Face Recognition Using Principal Component Analysis

Implementation and Concept Report

June 11, 2026

Abstract

This report details the theoretical foundations and software implementation of a face recognition system using the "Eigenfaces" approach. By leveraging Principal Component Analysis (PCA) on a dataset of human faces, high-dimensional image data is compressed into a lower-dimensional feature space. This allows for the efficient comparison and recognition of faces based on their projection weights. The implementation uses Python, OpenCV, and scikit-learn.

1 Introduction

Early attempts at computer-based facial recognition faced significant challenges due to limited computational power and memory constraints. In 1987, Sirovich and Kirby proposed that human face images could be represented as a weighted sum of a few "key pictures," which they called eigenpictures. Building on this, Turk and Pentland (1991) developed the "Eigenfaces" technique, utilizing these weights as characteristic features for facial recognition. This report breaks down how this linear algebra technique is implemented in modern Python.

2 Key Concepts and Mathematical Approach

2.1 Image Vectorization

In a computer, a grayscale picture is a matrix of pixels. To apply linear algebra techniques across an entire dataset, each image of size $r \times c$ is flattened into a single 1D vector of length $L = r \times c$. If the dataset contains M images, the entire dataset can be represented as an $L \times M$ matrix A, where each column is a single image.

2.2 Principal Component Analysis (PCA)

PCA is applied to matrix A to extract the principal components, which represent the key directions (or features) used to construct the images in the dataset. 1. **Mean Subtraction:** First, the mean vector a (the average face) is computed. A new matrix $C = A - a$ is formed by subtracting the mean face from every image vector. 2. **Covariance Matrix:** The covariance matrix is calculated as $S = C \cdot C^T$. 3. **Eigenvectors:** The eigenvectors

and eigenvalues of S are computed. The eigenvectors corresponding to the largest eigenvalues hold the most variance (information) and are retained. These top K eigenvectors are reshaped back into 2D images, known as **Eigenfaces**.

2.3 Weight Projection and Reconstruction

Any face picture can be projected onto the K eigenfaces using the vector dot-product. Denoting the matrix of eigenfaces as F (an $L \times K$ matrix), the projection yields a weight vector w. Conversely, a face can be mathematically approximated by calculating the weighted sum of the eigenfaces:

$$z = F \cdot w \tag{1}$$

Because the eigenface matrix F is constant for the given dataset, the weight vector w uniquely identifies an individual face.

3 Dataset Description

The implementation utilizes the **AT&T (ORL) Database of Faces**.

- **Total Images:** 400
- **Subjects (Classes):** 40 individuals, with 10 images per person.
- **Format:** PGM (Grayscale)
- **Image Shape:** 112×92 pixels ($10,304$ total pixels per image).

4 Code Implementation Overview

The software implementation is written in Python, utilizing `cv2` for image processing and `scikit-learn` for PCA.

4.1 Data Loading and Preprocessing

Images are read directly from a ZIP file using Python's `zipfile` module and decoded into pixel arrays using OpenCV (`cv2.imdecode`). To test the system's recognition capabilities, a specific training and testing split is established:

- **Training Set:** Classes 1 through 39 (excluding image 10 of class 39).
- **Test Set:** Image 10 of class 39 (a known person) and all images of class 40 (an unknown person).

The training images are flattened and stacked into a 2D NumPy array (`facematrix`).

4.2 Applying PCA

The `scikit-learn` library handles the heavy lifting of PCA. The `PCA().fit(facematrix)` function is called, which computes the mean face and the principal components. The algorithm is configured to keep the top $K = 50$ components to balance dimensional reduction with feature retention.

4.3 Computing Weights

Once the eigenfaces are extracted, the weight vectors for the entire training set are computed via matrix multiplication. In Python, this is executed efficiently using the `@` operator:

```
weights = eigenfaces @ (facematrix - pca.mean_).T
```

This single line subtracts the mean face from all images and projects them onto the eigenface space, resulting in a $K \times M$ weight matrix.

4.4 Facial Recognition (Matching)

To identify a query image, the system follows these steps: 1. The query image is flattened and mean-subtracted. 2. It is projected onto the eigenfaces to obtain its query weight vector. 3. The Euclidean distance (L2-norm) between the query weight vector and every weight vector in the training set is calculated using `np.linalg.norm`. 4. The training image with the minimum Euclidean distance (`np.argmin`) is selected as the "Best Match."

5 Conclusion

The Eigenface method provides a surprisingly effective approach to facial recognition relative to its mathematical simplicity. By mapping images into a lower-dimensional weight space, the system can quickly identify similarities using basic Euclidean distances. However, as noted by Turk and Pentland, the system's accuracy degrades when faced with variations in lighting, orientation, or size. While modern Convolutional Neural Networks (CNNs) have largely superseded Eigenfaces due to their robustness to these transformations, PCA remains a fundamental and highly illustrative machine learning technique.